

DEVELOPER & ARCHITECTURE REFERENCE

VolcanStorage

High-Performance Asynchronous GPU Asset Streaming and Hardware-Accelerated Decompression for Modern Vulkan Applications

Version: 1.0.0 Stable

Specification Compatibility: Vulkan 1.1, 1.2, 1.3, 1.4

Platforms Supported: Windows (x64, ARM64), Linux (x64, ARM64), macOS (Apple Silicon, x64 via MoltenVK)

License: MIT License (Open Source)

AUTHOR & MAINTAINER

Ahmet Enes Ay (Chiretallyn)

CORE CODEC

NVIDIA GDeflate (GLSL Compute SPIR-V)

TARGET AUDIENCE

Engine Programmers, Graphics Engineers

REPOSITORY

github.com/ahmetenesay/VolcanStorage

Table of Contents

1. Executive Overview & The NVMe Storage Dilemma	Section 1
2. VolcanStorage Architecture vs. Microsoft DirectStorage	Section 2
3. Graceful Hardware Downgrade Engine (Tiers 1–3)	Section 3
4. Core API Interfaces & Primitive Specifications	Section 4
5. GPU Decompression Pipeline (GDeflate GLSL / SPIR-V)	Section 5
6. Zero-Copy Streaming: Resizable BAR & UMA Architectures	Section 6
7. High-Performance Persistent Staging Pool Mechanics	Section 7
8. Complete Step-by-Step Integration Guide	Section 8
9. Building, Packaging, and Production Deployment	Section 9

1. Executive Overview

In modern high-fidelity interactive applications (Unreal Engine 5 Nanite/Lumen, streaming virtual textures, open-world asset streaming), storage I/O has replaced compute as the primary latency bottleneck. While modern Gen4 and Gen5 NVMe solid-state drives are capable of sustaining raw sequential read throughputs of **7,000 MB/s to 14,000 MB/s**, traditional CPU-centric game loading pipelines rarely exceed 1,500 MB/s.

The cause of this throughput degradation is three-fold:

- **OS Kernel Interrupts & File Context Switches:** Traditional synchronous or standard thread-pool `fread` / `ReadFile` calls require roundtrips into the OS scheduler, introducing microsecond-level stalls on thousands of individual asset chunks.
- **CPU Decompression Starvation:** Asset packages compressed with Deflate, Zstandard, or LZ4 saturate CPU cores during load phases, generating severe frame drops and thread contention with physics and audio systems.
- **Redundant Copy Hops:** Data moves from NVMe to OS page cache, into title memory, through a driver upload heap, and finally over PCI-e into VRAM, generating 3x to 4x memory bandwidth amplification.

VolcanStorage is a clean-slate, cross-platform C++20 storage runtime designed specifically for the **Vulkan API**. It solves these challenges by binding asynchronous NVMe disk requests directly to Vulkan GPU staging pipelines, decoding GDeflate-compressed assets on GPU compute units, and synchronizing transfers seamlessly via 64-bit Timeline Semaphores.

2. Architectural Comparison

DirectStorage was introduced by Microsoft to address these storage bottlenecks within the DirectX 12 ecosystem. However, Microsoft's implementation has several major limitations for cross-platform engine developers:

Architectural Dimension	Microsoft DirectStorage	VolcanStorage Runtime
Graphics API	DirectX 12 Only (Closed D3D12 device binding)	Vulkan 1.1 to 1.4 Native (Cross-vendor)
Operating Systems	Windows 10/11 & Xbox only	Windows, Linux, macOS (Apple Silicon / MoltenVK)
Source Availability	Closed-source proprietary binary (<code>dstorage.dll</code>)	100% Open Source (MIT License)
Compute Shaders	HLSL Shader Model 6.0 DXIL	GLSL #version 450 SPIR-V bytecode
Memory Architecture	Windows Upload Heaps (D3D12_HEAP_TYPE_UPLOAD)	ReBAR Zero-Copy, UMA Apple/ARM, Host-Visible Ring
Synchronization	<code>ID3D12Fence</code>	VkSemaphore (Timeline & Binary), Win32 Events
Fallback Path	Win32 Threadpool software decompression	Automated 3-Tier Hardware Capability Probe

3. Graceful Hardware Downgrade Engine

Not all target devices feature high-end discrete GPUs with Vulkan 1.4 and Resizable BAR. VolcanStorage implements a deterministic **Hardware Capability Probe** that discovers hardware features at initialization and selects the highest viable tier:

Tier 3: Modern GPU Pipeline (Vulkan 1.3 / 1.4)

- 3 64-bit Timeline Semaphores, Vulkan Synchronization2, GPU Compute GDeflate, ReBAR Zero-Copy Direct VRAM DMA.

12+
GB/s

Tier 2: Mainstream Pipeline (Vulkan 1.2 / Steam Deck / APUs)

- 2 Timeline Semaphores enabled, UMA unified memory path, multi-threaded CPU SIMD GDeflate fallback when GPU compute is occupied.

4–8
GB/s

Tier 1: Universal Compatibility (Vulkan 1.1 Legacy)

- 1 Standard binary `VkFence` / `VkSemaphore` pairs, POSIX `pread` / Win32 Overlapped, 64 MB host staging ring-buffer pool.

1.5–3
GB/s

Zero Crash Guarantee

If an end-user runs your application on an older device lacking `VK_KHR_timeline_semaphore` or `VK_KHR_synchronization2`, VulkanStorage automatically scales down internal queue dispatchers without triggering validation layer errors or device loss.

4. Core API Interfaces & Primitive Specifications

VolcanStorage provides an ergonomic C++20 interface that completely decouples storage queue management from your game engine's main render loop:

4.1. Factory Initialization (`IVolcanStorageFactory`)

```
#include <volcanstorage/volcanstorage.h>

volcan::FactoryDesc desc{};
desc.Device = myVulkanDevice;
desc.PhysicalDevice = myPhysicalDevice;
desc.DefaultStagingBufferSize = 64 * 1024 * 1024; // 64 MB staging ring pool

volcan::IVolcanStorageFactory* factory = nullptr;
VkResult res = volcan::CreateVolcanStorageFactory(&desc, &factory);
if (res == VK_SUCCESS) {
    const volcan::VolcanDeviceCapabilities& caps = factory->GetCapabilities();
    printf("Tier: %d, ReBAR: %s, Timelines: %s\n",
        caps.Tier,
        caps.SupportsReBAR ? "YES" : "NO",
        caps.SupportsTimelineSemaphores ? "YES" : "NO");
}
```

4.2. Asynchronous File Access (`IVolcanStorageFile`)

Files are opened with native operating system non-blocking flags (`FILE_FLAG_OVERLAPPED` | `FILE_FLAG_NO_BUFFERING` on Windows, `O_DIRECT` / `pread` on Linux):

```
volcan::IVolcanStorageFile* assetFile = nullptr;
res = factory->OpenFile(L"Content/Pak/Textures_4K.volcan", &assetFile);
uint64_t fileSize = assetFile->GetFileSize();
```

4.3. Creating Storage Queues (`IVolcanStorageQueue`)

Queues are categorized by priority and capacity. A typical architecture maintains two queues: one high-priority queue for immediate camera frustum streaming, and one normal-priority queue for background asset pre-fetching.

```
volcan::QueueDesc qDesc{};
qDesc.Capacity = 2048; // Up to 2048 requests in flight
qDesc.QueuePriority = volcan::Priority::Realtime; // High execution priority
qDesc.Name = "StreamingTextureQueue";

volcan::IVolcanStorageQueue* streamQueue = nullptr;
factory->CreateQueue(&qDesc, &streamQueue);
```

4.4. The Request Descriptor (`volcan::Request`)

Each individual load request specifies its source file, offset, compressed/uncompressed sizes, and destination Vulkan resource:

Field	Type	Description
<code>File</code>	<code>IVolcanStorageFile*</code>	Source file handle opened via the factory.
<code>SourceOffset</code>	<code>uint64_t</code>	Byte offset within the archive file.
<code>SourceSize</code>	<code>uint32_t</code>	Compressed byte length on disk.
<code>UncompressedSize</code>	<code>uint32_t</code>	Expected decompressed payload size.
<code>DestType</code>	<code>DestinationType</code>	<code>Buffer</code> , <code>Image</code> , or <code>Memory</code> .
<code>Destination.Buffer</code>	<code>VkBuffer, Offset</code>	Target Vulkan buffer and destination byte offset.
<code>Destination.Image</code>	<code>VkImage, Extent, Mip</code>	Target texture, dimensions, and mip level.

5. GPU Decompression Pipeline (GDeflate SPIR-V)

VolcanStorage standardizes on **NVIDIA GDeflate**, an open, royalty-free byte-stream format specifically engineered for massively parallel SIMD/SIMT architectures:

- **Tile-Based Partitioning:** Uncompressed assets are broken into discrete 64 KB tiles. Each tile is compressed independently into the bitstream.
- **Branchless Prefix-Code Decoding:** Compute shader workgroups (typically 128 or 256 threads) process independent bitstreams simultaneously, eliminating divergence stalls.
- **GLSL Compute Implementation:** The primary shader is compiled from `shaders/GDeflate.comp` using `glslc --target-env=vulkan1.1` into standard SPIR-V, ensuring execution across AMD, NVIDIA, Intel, and Apple GPUs.

Automatic Image Layout Transitions

When `DestinationType::Image` is targeted, VolcanStorage's internal transfer queue automatically executes barrier transitions: `VK_IMAGE_LAYOUT_UNDEFINED` → `VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL` → `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`. Your engine can sample the texture immediately after the timeline semaphore signals!

6. Zero-Copy Streaming: Resizable BAR & UMA

Legacy graphics architectures limited the CPU-to-GPU PCI Express transfer aperture to 256 MB. With modern **Resizable BAR (ReBAR)** and AMD **Smart Access Memory (SAM)**, the entire VRAM pool (e.g. 16 GB to 24 GB) is directly addressable in host memory space.

VolcanStorage dynamically checks for:

```
VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT | VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT
```

When this combination is detected, VolcanStorage maps the destination buffer directly to the OS disk I/O subsystem. Data read from the NVMe controller is DMA-transferred **directly into VRAM** across PCI Express Gen4/5, completely bypassing host DRAM staging copies.

Memory Flow Paths:

A

Zero-Copy Path (ReBAR & Apple Silicon / AMD APU UMA):

NVMe → Direct PCI-e DMA → Device Local VRAM (Zero staging overhead)

Direct DMA

B

Standard Discrete GPU Path:

NVMe → 64 MB Persistent Ring Buffer → GPU Copy Engine → VRAM

Ring Buffered

7. Persistent Staging Pool Mechanics

A critical mistake in naive Vulkan storage implementations is allocating temporary `VkDeviceMemory` for every streaming batch. Frequent allocations trigger OS page table updates, GPU memory fragmentation, and severe frame hitching.

VolcanStorage allocates a single **persistent 64 MB ring buffer** (configurable via `DefaultStagingBufferSize`) at factory initialization:

- Requests acquire sub-allocated offsets into the ring buffer via atomic increments.
- When transfers complete and are retired by GPU semaphores, ring buffer segments are reclaimed in constant time $O(1)$.
- Zero runtime allocations occur during intense open-world streaming!

8. Complete Step-by-Step Integration Guide

The following example illustrates a complete end-to-end streaming cycle with monotonic Timeline Semaphore synchronization:

```

// 1. Create a 64-bit Timeline Semaphore for storage sync
VkSemaphoreTypeCreateInfo timelineCreateInfo{
    .sType = VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO,
    .semaphoreType = VK_SEMAPHORE_TYPE_TIMELINE,
    .initialValue = 0
};
VkSemaphoreCreateInfo semInfo{
    .sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
    .pNext = &timelineCreateInfo
};
VkSemaphore storageTimeline;
vkCreateSemaphore(device, &semInfo, nullptr, &storageTimeline);

// 2. Enqueue asynchronous load requests
volcan::Request req{};
req.File = worldArchive;
req.SourceOffset = 1048576; // Byte offset in pak
req.SourceSize = 262144;    // 256 KB compressed
req.UncompressedSize = 1048576; // 1 MB uncompressed
req.DestType = volcan::DestinationType::Buffer;
req.Destination.Buffer.TargetBuffer = terrainVBO;
req.Destination.Buffer.Offset = 0;
streamQueue->EnqueueRequest(&req);

// 3. Enqueue a fence signal at monotonic value 42
uint64_t expectedSyncValue = 42;
streamQueue->EnqueueSignalTimeline(storageTimeline, expectedSyncValue);

// 4. Submit the batch to disk and GPU transfer queues
streamQueue->Submit();

// 5. In your render loop: Wait or instruct graphics command buffer to wait on GPU
VkTimelineSemaphoreSubmitInfo timelineSubmit{
    .sType = VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO,
    .waitSemaphoreValueCount = 1,
    .pWaitSemaphoreValues = &expectedSyncValue
};
// Submit your draw call - it executes the instant loading finishes!

```

9. Building, Packaging & Production Deployment

VolcanStorage integrates seamlessly into standard CMake build environments. It requires a C++20 compliant compiler (MSVC v143+, GCC 12+, Clang 15+) and the Vulkan SDK (version 1.1 or newer).

9.1. Integration via CMake

```
# In your engine's CMakeLists.txt:
add_subdirectory(third_party/VolcanStorage)
target_link_libraries(MyGameEngine PRIVATE VolcanStorage)
```

9.2. Automated Packaging Tool (`scripts/package_release.ps1`)

The repository includes an automated packaging pipeline that packages release zip archives containing headers, binary libraries, SPIR-V shaders, documentation, and notices:

```
# Package all locally built platforms into dist/
pwsh -ExecutionPolicy Bypass -File scripts/package_release.ps1 -All

# Or build individual platforms:
pwsh -ExecutionPolicy Bypass -File scripts/package_release.ps1 -Platform Windows-x64
pwsh -ExecutionPolicy Bypass -File scripts/package_release.ps1 -Platform Linux-arm64
```

9.3. Distribution Package Architecture

Directory / Artifact	Description
<code>bin/</code>	Compiled runtime binaries (<code>volcanstorage.dll</code> , <code>.lib</code> , <code>.so</code> , <code>.dylib</code>) and <code>shaders/GDeflate.spv</code>
<code>include/volcanstorage/</code>	Public C++20 headers (<code>volcanstorage.h</code>)
<code>docs/</code>	Developer guide and architectural specifications
<code>LICENSE</code> & <code>NOTICES.txt</code>	MIT open source license & third-party attributions (Microsoft, NVIDIA)